

5. 增删改查操作

1. 本节目标

- 熟练使用SQL语句对数据库进行CRUD操作
- 掌握聚合函数的使用
- 掌握分组查询以及对分组结果进行过滤
- 了解内置函数

2. CRUD简介

CRUD是对数据库中的记录进行基本的增删改查操作：

- Create (创建)
- Retrieve (读取)
- Update (更新)
- Delete (删除)

3. Create 新增

3.1 语法

```
1 INSERT [INTO] table_name
2     [(column [, column] ...)]
3 VALUES
4     (value_list) [, (value_list)] ...
5
6 value_list: value, [, value] ...
```

3.2 示例：

```
1 # 创建一个用于演示的表
2 create table users (
3     id bigint,
4     name varchar(20) comment '用户名'
```

5);

3.2.1 单行数据全列插入

- **value_list** 中值的数量必须和定义表的列的数量及顺序一致

```
1 # 插入第一条记录
2 mysql> insert into users values (1, '张三');
3 Query OK, 1 row affected (0.02 sec)
4
5 # 插入第二条记录
6 mysql> insert into users values (2, '李四');
7 Query OK, 1 row affected (0.01 sec)
8
9 # 查询结果
10 mysql> select * from users;
11 +-----+-----+
12 | id    | name |
13 +-----+-----+
14 |     1 | 张三 |
15 |     2 | 李四 |
16 +-----+-----+
17 2 rows in set (0.00 sec)
```

3.2.2 单行数据指定列插入

- **value_list** 中值的数量必须和指定列数量及顺序一致

```
1 # 指定了具体要插入的列
2 mysql> insert into users(id, name) values (3, '王五');
3 Query OK, 1 row affected (0.01 sec)
4
5 mysql> select * from users;
6 +-----+-----+
7 | id    | name |
8 +-----+-----+
9 |     1 | 张三 |
10 |     2 | 李四 |
11 |     3 | 王五 |
12 +-----+-----+
13 3 rows in set (0.00 sec)
```

3.2.3 多行数据指定列插入

- 在一条INSERT语句中也可以指定多个value_list，实现一次插入多行数据

```
1 # 每个value_list表示一行数据
2 mysql> insert into users(id, name) values (4, '赵六'), (5, '钱七');
3 Query OK, 2 rows affected (0.00 sec)
4 Records: 2 Duplicates: 0 Warnings: 0
5
6 mysql> select * from users;
7 +-----+-----+
8 | id    | name |
9 +-----+-----+
10 | 1     | 张三 |
11 | 2     | 李四 |
12 | 3     | 王五 |
13 | 4     | 赵六 |
14 | 5     | 钱七 |
15 +-----+-----+
16 5 rows in set (0.00 sec)
```

4. Retrieve 检索

4.1 语法

```
1 SELECT
2     [DISTINCT]
3     select_expr [, select_expr] ...
4     [FROM table_references]
5     [WHERE where_condition]
6     [GROUP BY {col_name | expr}, ...]
7     [HAVING where_condition]
8     [ORDER BY {col_name | expr} [ASC | DESC], ... ]
9     [LIMIT {[offset], row_count | row_count OFFSET offset}]
```

4.2 示例：

4.2.1 构造数据

```
1 -- 创建表结构
```

```

2 CREATE TABLE exam (
3     id BIGINT,
4     name VARCHAR(20) COMMENT '同学姓名',
5     chinese float COMMENT '语文成绩',
6     math float COMMENT '数学成绩',
7     english float COMMENT '英语成绩'
8 );
9 -- 插入测试数据
10 INSERT INTO exam (name, chinese, math, english) VALUES
11 (1, '唐三藏', 67, 98, 56),
12 (2, '孙悟空', 87, 78, 77),
13 (3, '猪悟能', 88, 98, 90),
14 (4, '曹孟德', 82, 84, 67),
15 (5, '刘玄德', 55, 85, 45),
16 (6, '孙权', 70, 73, 78),
17 (7, '宋公明', 75, 65, 30);
18 Query OK, 7 rows affected (0.00 sec)
19 Records: 7 Duplicates: 0 Warnings: 0

```

4.3 Select

4.3.1 全列查询

- 查询所有记录

```

1 # 使用 * 可以查询表中所有列的值
2 mysql> select * from exam;
3 +-----+-----+-----+-----+
4 | id | name   | chinese | math | english |
5 +-----+-----+-----+-----+
6 | 1 | 唐三藏 | 67      | 98   | 56      |
7 | 2 | 孙悟空 | 87      | 78   | 77      |
8 | 3 | 猪悟能 | 88      | 98   | 90      |
9 | 4 | 曹孟德 | 82      | 84   | 67      |
10 | 5 | 刘玄德 | 55      | 85   | 45      |
11 | 6 | 孙权   | 70      | 73   | 78      |
12 | 7 | 宋公明 | 75      | 65   | 30      |
13 +-----+-----+-----+-----+
14 7 rows in set (0.00 sec)

```

4.3.2 指定列查询

- 查询所有人的编号、姓名和语文成绩

```

1 mysql> select id, name, chinese from exam;
2 +-----+
3 | id | name   | chinese |
4 +-----+
5 | 1 | 唐三藏 | 67      |
6 | 2 | 孙悟空 | 87      |
7 | 3 | 猪悟能 | 88      |
8 | 4 | 曹孟德 | 82      |
9 | 5 | 刘玄德 | 55      |
10 | 6 | 孙权   | 70      |
11 | 7 | 宋公明 | 75      |
12 +-----+
13 7 rows in set (0.00 sec)

```

在select后面的查询列表中指定希望查询的列，可以是一个也可以是多个，中间用逗号隔开
指定列的顺序与表结构中的列的顺序无关

4.3.3 查询字段为表达式

- 常量表达式

```

1 # 表达式本身就是一个常
2 mysql> select id, name, 10 from exam;
3 +-----+
4 | id | name   | 10      |
5 +-----+
6 | 1 | 唐三藏 | 10      |
7 | 2 | 孙悟空 | 10      |
8 | 3 | 猪悟能 | 10      |
9 | 4 | 曹孟德 | 10      |
10 | 5 | 刘玄德 | 10      |
11 | 6 | 孙权   | 10      |
12 | 7 | 宋公明 | 10      |
13 +-----+
14 7 rows in set (0.00 sec)
15
16 # 也可以是常量的运算
17 mysql> select id, name, 10 + 1 from exam;
18 +-----+
19 | id | name   | 10 + 1  |
20 +-----+
21 | 1 | 唐三藏 | 11      |
22 | 2 | 孙悟空 | 11      |
23 | 3 | 猪悟能 | 11      |
24 | 4 | 曹孟德 | 11      |

```

```

25 | 5 | 刘玄德 | 11 |
26 | 6 | 孙权 | 11 |
27 | 7 | 宋公明 | 11 |
28 +-----+
29 7 rows in set (0.01 sec)

```

• 把所有学生的语文成绩加10分

```

1 # 表达式中包含一个字段
2 mysql> select id, name, chinese + 10 from exam;
3 +-----+
4 | id | name | chinese + 10 |
5 +-----+
6 | 1 | 唐三藏 | 77 |
7 | 2 | 孙悟空 | 97 |
8 | 3 | 猪悟能 | 98 |
9 | 4 | 曹孟德 | 92 |
10 | 5 | 刘玄德 | 65 |
11 | 6 | 孙权 | 80 |
12 | 7 | 宋公明 | 85 |
13 +-----+
14 7 rows in set (0.00 sec)

```

• 计算所有学生语文、数学和英语成绩的总分

```

1 # 表达式包含多个字段
2 mysql> select id, name, chinese + math + english from exam;
3 +-----+
4 | id | name | chinese + math + english |
5 +-----+
6 | 1 | 唐三藏 | 221 |
7 | 2 | 孙悟空 | 242 |
8 | 3 | 猪悟能 | 276 |
9 | 4 | 曹孟德 | 233 |
10 | 5 | 刘玄德 | 185 |
11 | 6 | 孙权 | 221 |
12 | 7 | 宋公明 | 170 |
13 +-----+
14 7 rows in set (0.00 sec)

```

4.3.4 为查询结果指定别名

4.3.4.1 语法

```
1 SELECT column [AS] alias_name [, ...] FROM table_name;
```

AS可以省略，别名如果包含空格必须用单引号包裹

4.3.4.2 示例

- 为总分这一列指定别名

```
1 mysql> select id, name, chinese + math + english as 总分 from exam;
2 +-----+
3 | id | name   | 总分 | # 表头以别名显示
4 +-----+
5 | 1 | 唐三藏 | 221 |
6 | 2 | 孙悟空 | 242 |
7 | 3 | 猪悟能 | 276 |
8 | 4 | 曹孟德 | 233 |
9 | 5 | 刘玄德 | 185 |
10 | 6 | 孙权   | 221 |
11 | 7 | 宋公明 | 170 |
12 +-----+
13 7 rows in set (0.00 sec)
```

4.3.5 结果去重查询

- 查询当前所的数学成绩

```
1 # 通过观察有两条98的记录
2 mysql> select math from exam;
3 +-----+
4 | math |
5 +-----+
6 | 98 | # 第一条
7 | 78 |
8 | 98 | # 第二条
9 | 84 |
10 | 85 |
11 | 73 |
12 | 65 |
13 +-----+
14 7 rows in set (0.00 sec)
```

2. 在结果集中去除重复记录，可以使用DISTINCT

```
1 # 去重查询
2 mysql> select distinct math from exam;
3 +-----+
4 | math |
5 +-----+
6 |    98 |
7 |    78 |
8 |    84 |
9 |    85 |
10 |    73 |
11 |    65 |
12 +-----+
13 6 rows in set (0.00 sec)
```

使用DISTINCT去重时，只有查询列表中所有列的值都相同才会判定为重复

注意：

- 查询时不加限制条件会返回表中所有结果，如果表中的数据量过大，会把服务器的资源消耗殆尽
- 在生产环境不要使不加限制条件的查询

4.4 Where 条件查询

4.4.1 语法

```
1 SELECT
2     select_expr [, select_expr] ... [FROM table_references]
3     WHERE where_condition
```

4.4.2 比较运算符

运算符	说明
>, >=, <, <=	大于，大于等于，小于，小于等于
=	等于，对于NULL的比较不安全，比如NULL = NULL结果还是NULL
<=>	等于，对于NULL的比较是安全的，比如NULL <=> NULL结果是TRUE(1)

!=, <>	不等于
value BETWEEN a0 AND a1	范围匹配, [a0, a1], 如果a0 <= value <= a1, 返回TRUE或1, NOT BETWEEN则取反
value IN (option, ...)	如果value 在option列表中, 则返回TRUE(1), NOT IN则取反
IS NULL	是NULL
IS NOT NULL	不是NULL
LIKE	模糊匹配, % 表示任意多个(包括0个)字符; _ 表示任意一个字符, NOT LIKE则取反

4.4.3 逻辑运算符

运算符	说明
AND	多个条件必须都为 TRUE(1), 结果才是 TRUE(1)
OR	任意一个条件为 TRUE(1), 结果为 TRUE(1)
NOT	条件为 TRUE(1), 结果为 FALSE(0)

4.4.4 示例

4.4.4.1 基本查询

- 查询英语不及格的同学及英语成绩 (< 60)

```

1 mysql> select name, english from exam where english < 60;
2 +-----+-----+
3 | name   | english |
4 +-----+-----+
5 | 唐三藏 |      56 |
6 | 刘玄德 |      45 |
7 | 宋公明 |      30 |
8 +-----+-----+
9 3 rows in set (0.00 sec)

```

- 查询语文成绩高于英语成绩的同学

```

1 mysql> select name, chinese, english from exam where chinese > english;
2 +-----+-----+-----+
3 | name   | chinese | english |

```

```

4 +-----+-----+-----+
5 | 唐三藏 |      67 |      56 |
6 | 孙悟空 |      87 |      77 |
7 | 曹孟德 |      82 |      67 |
8 | 刘玄德 |      55 |      45 |
9 | 宋公明 |      75 |      30 |
10 +-----+-----+-----+
11 5 rows in set (0.00 sec)

```

- 总分在 200 分以下的同学

```

1 mysql> select name, chinese + math + english as 总分 from exam where chinese +
2 math + english < 200;
3 +-----+-----+
4 | name | 总分 |
5 +-----+-----+
6 | 刘玄德 | 185 |
7 | 宋公明 | 170 |
8 +-----+-----+
9 2 rows in set (0.00 sec)

```

4.4.4.2 AND和OR

- 查询语文成绩大于80分且英语成绩大于80分的同学

```

1 mysql> select * from exam where chinese > 80 and english > 80;
2 +-----+-----+-----+-----+
3 | id | name | chinese | math | english |
4 +-----+-----+-----+-----+
5 | 3 | 猪悟能 | 88 | 98 | 90 |
6 +-----+-----+-----+-----+
7 1 row in set (0.00 sec)

```

- 查询语文成绩大于80分或英语成绩大于80分的同学

```

1 mysql> select * from exam where chinese > 80 OR english > 80;
2 +-----+-----+-----+-----+
3 | id | name | chinese | math | english |
4 +-----+-----+-----+-----+
5 | 2 | 孙悟空 | 87 | 78 | 77 |
6 | 3 | 猪悟能 | 88 | 98 | 90 |
7 | 4 | 曹孟德 | 82 | 84 | 67 |

```

```
8 +-----+-----+-----+-----+
9 3 rows in set (0.00 sec)
```

- 观察AND和OR的优先级

```
1 mysql> select * from exam where chinese > 80 or math > 70 and english > 70;
2 mysql> select * from exam where (chinese > 80 or math > 70) and english > 70;
```

4.4.4.3 范围查询

- 语文成绩在 [80, 90] 分的同学及语文成绩

```
1 # 使用BETWEEN AND 实现
2 mysql> select name, chinese from exam where chinese between 80 and 90;
3 # 使用 AND 实现
4 mysql> select name, chinese from exam where chinese >= 80 and chinese <= 90;
5 +-----+-----+
6 | name   | chinese |
7 +-----+-----+
8 | 孙悟空 |      87 |
9 | 猪悟能 |      88 |
10 | 曹孟德 |      82 |
11 +-----+-----+
12 3 rows in set (0.00 sec)
13
14
```

- 数学成绩是 78 或者 79 或者 98 或者 99 分的同学及数学成绩

```
1 # 使用IN实现
2 mysql> select name, math from exam where math in (78, 79, 98, 99);
3 # 使用OR实现
4 mysql> select name, math from exam where math = 78 or math = 79 or math = 98 or
   math = 99;
5
6 +-----+-----+
7 | name   | math |
8 +-----+-----+
9 | 唐三藏 |   98 |
10 | 孙悟空 |   78 |
11 | 猪悟能 |   98 |
12 +-----+-----+
```

4.4.4.4 模糊查询

- 查询所有姓孙的同学

```
1 mysql> select * from exam where name like '孙%';
2 +-----+
3 | id | name   | chinese | math | english |
4 +-----+
5 | 2 | 孙悟空 | 87      | 78   | 77      |
6 | 6 | 孙权   | 70      | 73   | 78      |
7 +-----+
8 2 rows in set (0.00 sec)
```

- 查询姓孙且姓名共有两个字同学

```
1 mysql> select * from exam where name like '孙_';
2 +-----+
3 | id | name   | chinese | math | english |
4 +-----+
5 | 6 | 孙权   | 70      | 73   | 78      |
6 +-----+
7 1 row in set (0.00 sec)
```

4.4.4.5 NULL的查询

- 构造数据

```
1 # 写入一条数据，英语成绩为NULL
2 insert into exam values (8, '张飞', 27, 0, NULL);
```

- 查询英语成绩为NULL的记录

```
1 # 使用is null
2 mysql> select * from exam where english is null;
3 +-----+
4 | id | name   | chinese | math | english |
5 +-----+
6 | 8 | 张飞   | 27      | 0    | NULL    |
```

```
7 +-----+
8 1 row in set (0.00 sec)
```

- 查询英语成绩不为NULL的记录

```
1 # 使用is not null
2 mysql> select * from exam where english is not null;
3 +-----+
4 | id | name   | chinese | math | english |
5 +-----+
6 | 1 | 唐三藏 | 67      | 98   | 56      |
7 | 2 | 孙悟空 | 87      | 78   | 77      |
8 | 3 | 猪悟能 | 88      | 98   | 90      |
9 | 4 | 曹孟德 | 82      | 84   | 67      |
10 | 5 | 刘玄德 | 55      | 85   | 45      |
11 | 6 | 孙权   | 70      | 73   | 78      |
12 | 7 | 宋公明 | 75      | 65   | 30      |
13 +-----+
14 7 rows in set (0.00 sec)
```

- NULL与其他值进行运算结果为NULL

```
1 # 观察结果中的总分
2 mysql> select name, chinese + math + english as 总分 from exam;
3 +-----+
4 | name   | 总分 |
5 +-----+
6 | 唐三藏 | 221  |
7 | 孙悟空 | 242  |
8 | 猪悟能 | 276  |
9 | 曹孟德 | 233  |
10 | 刘玄德 | 185  |
11 | 孙权   | 221  |
12 | 宋公明 | 170  |
13 | 张飞   | NULL |
14 +-----+
15 8 rows in set (0.00 sec)
```

注意

- WHERE条件中可以使用表达式，但不能使用别名
- AND的优先级高于OR，在同时使用时，建议使用小括号()包裹优先执行的部分

- 过滤NULL时不要使用等于号(=)与不等于号(!=, <>)
- NULL与任何值运算结果都为NULL

4.5 Order by 排序

4.5.1 语法

```
1 -- ASC 为升序 (从小到大)
2 -- DESC 为降序 (从大到小)
3 -- 默认为 ASC
4 SELECT ... FROM table_name [WHERE ...] ORDER BY {col_name | expr} [ASC |
DESC], ... ;
```

4.5.2 示例

- 按数学成绩从低到高排序(升序)

```
1 mysql> select name, math from exam order by math asc;
2 +-----+-----+
3 | name   | math |
4 +-----+-----+
5 | 张飞   | 0    |
6 | 宋公明 | 65   |
7 | 孙权   | 73   |
8 | 孙悟空 | 78   |
9 | 曹孟德 | 84   |
10 | 刘玄德 | 85   |
11 | 唐三藏 | 98   |
12 | 猪悟能 | 98   |
13 +-----+-----+
14 8 rows in set (0.00 sec)
```

- 按语文成绩从高到低排序(降序)

```
1 mysql> select name, chinese from exam order by chinese desc;
2 +-----+-----+
3 | name   | chinese |
4 +-----+-----+
5 | 猪悟能 | 88      |
6 | 孙悟空 | 87      |
7 | 曹孟德 | 82      |
```

```

8 | 宋公明 | 75 |
9 | 孙权 | 70 |
10 | 唐三藏 | 67 |
11 | 刘玄德 | 55 |
12 | 张飞 | 27 |
13 +-----+
14 8 rows in set (0.00 sec)

```

- 按英语成绩从高到低排序

```

1 mysql> select name, english from exam order by english desc;
2 +-----+
3 | name | english |
4 +-----+
5 | 猪悟能 | 90 |
6 | 孙权 | 78 |
7 | 孙悟空 | 77 |
8 | 曹孟德 | 67 |
9 | 唐三藏 | 56 |
10 | 刘玄德 | 45 |
11 | 宋公明 | 30 |
12 | 张飞 | NULL | # NULL被看做比任何值都小
13 +-----+
14 8 rows in set (0.00 sec)

```

- 查询同学各门成绩，依次按数学降序，英语升序，语文升序的方式显示

```

1 mysql> select name, math, english, chinese from exam order by math desc,
2 english asc, chinese asc;
3 +-----+
4 | name | math | english | chinese |
5 +-----+
6 | 唐三藏 | 98 | 56 | 67 |
7 | 猪悟能 | 98 | 90 | 88 |
8 | 刘玄德 | 85 | 45 | 55 |
9 | 曹孟德 | 84 | 67 | 82 |
10 | 孙悟空 | 78 | 77 | 87 |
11 | 孙权 | 73 | 78 | 70 |
12 | 宋公明 | 65 | 30 | 75 |
13 | 张飞 | 0 | NULL | 27 |
14 +-----+
15 8 rows in set (0.00 sec)

```

- 查询同学及总分，由高到低排序

```
1 mysql> select name, chinese + math + english from exam order by chinese + math
+ english desc;
2 +-----+-----+
3 | name   | chinese + math + english |
4 +-----+-----+
5 | 猪悟能 |                          276 |
6 | 孙悟空 |                          242 |
7 | 曹孟德 |                          233 |
8 | 唐三藏 |                          221 |
9 | 孙权   |                          221 |
10 | 刘玄德 |                          185 |
11 | 宋公明 |                          170 |
12 | 张飞   |                          NULL |
13 +-----+-----+
14 8 rows in set (0.00 sec)
```

- 可以使用列的别名进行排序

```
1 mysql> select name, chinese + math + english as 总分 from exam order by 总分
desc;
2 +-----+-----+
3 | name   | 总分 |
4 +-----+-----+
5 | 猪悟能 | 276 |
6 | 孙悟空 | 242 |
7 | 曹孟德 | 233 |
8 | 唐三藏 | 221 |
9 | 孙权   | 221 |
10 | 刘玄德 | 185 |
11 | 宋公明 | 170 |
12 | 张飞   | NULL |
13 +-----+-----+
14 8 rows in set (0.00 sec)
```

- 所有英语成绩不为NULL的同学，按语文成绩从高到低排序

```
1 mysql> select * from exam where english is not null order by chinese desc;
2 +-----+-----+-----+-----+-----+
3 | id | name   | chinese | math | english |
4 +-----+-----+-----+-----+-----+
```


5	3	猪悟能	88	98	90
6	2	孙悟空	87	78	77
7	4	曹孟德	82	84	67
8	7	宋公明	75	65	30
9	6	孙权	70	73	78
10	1	唐三藏	67	98	56
11	5	刘玄德	55	85	45

```

12 +-----+
13 7 rows in set (0.00 sec)

```

注意

- 查询中没有ORDER BY子句，返回的顺序是未定义的，永远不要依赖这个顺序
- ORDER BY子句中可以使用列的别名进行排序
- NULL进行排序时，视为比任何值都小，升序出现在最上面，降序出现在最下面

4.6 分页查询

4.6.1 语法

```

1 -- 起始下标为 0
2 -- 从 0 开始，筛选 num 条结果
3 SELECT ... FROM table_name [WHERE ...] [ORDER BY ...] LIMIT num;
4 -- 从 start 开始，筛选 num 条结果
5 SELECT ... FROM table_name [WHERE ...] [ORDER BY ...] LIMIT start, num;
6 -- 从 start 开始，筛选 num 条结果，比第二种用法更明确，建议使用
7 SELECT ... FROM table_name [WHERE ...] [ORDER BY ...] LIMIT num OFFSET start;

```

4.6.2 示例

```

1 # 查询第一页数据
2 mysql> select * from exam order by id asc limit 0, 3;
3 +-----+
4 | id | name   | chinese | math | english |
5 +-----+
6 | 1  | 唐三藏 | 67      | 98   | 56      |
7 | 2  | 孙悟空 | 87      | 78   | 77      |
8 | 3  | 猪悟能 | 88      | 98   | 90      |
9 +-----+
10 3 rows in set (0.00 sec)
11
12 # 查询第二页数据

```

```
13 mysql> select * from exam order by id asc limit 3, 3;
```

```
14 +-----+
15 | id | name   | chinese | math | english |
16 +-----+
17 | 4 | 曹孟德 | 82      | 84   | 67      |
18 | 5 | 刘玄德 | 55      | 85   | 45      |
19 | 6 | 孙权   | 70      | 73   | 78      |
20 +-----+
```

```
21 3 rows in set (0.00 sec)
```

```
22
```

23 # 查询第三页数据，没有达到limit的条数限制，也不会有任何影响，有多少条就显示多少条

```
24 mysql> select * from exam order by id asc limit 6, 3;
```

```
25 +-----+
26 | id | name   | chinese | math | english |
27 +-----+
28 | 7 | 宋公明 | 75      | 65   | 30      |
29 | 8 | 张飞   | 27      | 0    | NULL    |
30 +-----+
```

```
31 2 rows in set (0.00 sec)
```

5. Update 修改

5.1 语法

```
1 UPDATE [LOW_PRIORITY] [IGNORE] table_reference
2     SET assignment [, assignment] ...
3     [WHERE where_condition]
4     [ORDER BY ...]
5     [LIMIT row_count]
```

- 对符合条件的结果进行列值更新

5.2 示例

- 将孙悟空同学的数学成绩变更为 80 分

```
1 # 查看原始数据
2 mysql> select * from exam where name = '孙悟空';
3 +-----+
4 | id | name   | chinese | math | english |
5 +-----+
6 | 2 | 孙悟空 | 87      | 78   | 77      |
```

```

7 +-----+-----+-----+-----+
8 1 row in set (0.00 sec)
9
10 # 更新操作
11 mysql> update exam set math = 80 where name = '孙悟空';
12 Query OK, 1 row affected (0.01 sec)
13 Rows matched: 1   Changed: 1   Warnings: 0
14
15 # 查看结果，数学成绩更新成功
16 mysql> select * from exam where name = '孙悟空';
17 +-----+-----+-----+-----+
18 | id | name   | chinese | math | english |
19 +-----+-----+-----+-----+
20 | 2  | 孙悟空 |      87 |   80 |        77 |
21 +-----+-----+-----+-----+
22 1 row in set (0.00 sec)

```

- 将曹孟德同学的数学成绩变更为 60 分，语文成绩变更为 70 分

```

1 # 查看原始数据
2 mysql> select name, math, chinese from exam where name = '曹孟德';
3 +-----+-----+-----+
4 | name   | math | chinese |
5 +-----+-----+-----+
6 | 曹孟德 |   84 |      82 |
7 +-----+-----+-----+
8 1 row in set (0.00 sec)
9
10 # 更新操作
11 mysql> update exam set math = 60, chinese = 70 where name = '曹孟德';
12 Query OK, 1 row affected (0.01 sec)
13 Rows matched: 1   Changed: 1   Warnings: 0
14
15 # 查看结果
16 mysql> select name, math, chinese from exam where name = '曹孟德';
17 +-----+-----+-----+
18 | name   | math | chinese |
19 +-----+-----+-----+
20 | 曹孟德 |   60 |      70 |
21 +-----+-----+-----+
22 1 row in set (0.00 sec)

```

- 将总成绩倒数前三的 3 位同学的数学成绩加上 30 分

```

1 # 查看原始数据
2 mysql> select name, math,chinese + math + english as 总分 from exam where
  chinese + math + english is not null order by 总分 asc limit 3;
3 +-----+-----+-----+
4 | name   | math | 总分 |
5 +-----+-----+-----+
6 | 宋公明 |   65 |  170 |
7 | 刘玄德 |   85 |  185 |
8 | 曹孟德 |   60 |  197 |
9 +-----+-----+-----+
10 3 rows in set (0.00 sec)
11
12 # 更新操作
13 mysql> update exam set math = math +30 where chinese + math + english is not
  null order by chinese + math + english asc limit 3;
14 Query OK, 3 rows affected (0.01 sec)
15 Rows matched: 3   Changed: 3   Warnings: 0
16
17 # 查看结果
18 mysql> select name, math, chinese + math + english as 总分 from exam where name
  in ('宋公明','刘玄德','曹孟德');
19 +-----+-----+-----+
20 | name   | math | 总分 |
21 +-----+-----+-----+
22 | 曹孟德 |   90 |  227 |
23 | 刘玄德 |  115 |  215 |
24 | 宋公明 |   95 |  200 |
25 +-----+-----+-----+
26 3 rows in set (0.00 sec)
27
28 # 修改后总成绩倒数前三的 3 位同学和数据成绩
29 mysql> select name, math,chinese + math + english as 总分 from exam where
  chinese + math + english is not null order by 总分 asc limit 3;
30 +-----+-----+-----+
31 | name   | math | 总分 |
32 +-----+-----+-----+
33 | 宋公明 |   95 |  200 |
34 | 刘玄德 |  115 |  215 |
35 | 唐三藏 |   98 |  221 |
36 +-----+-----+-----+
37 3 rows in set (0.00 sec)

```

- 将所有同学的语文成绩更新为原来的 2 倍

```

1 # 查看原始数据

```

```

2 mysql> select * from exam;
3 +-----+-----+-----+-----+
4 | id | name   | chinese | math | english |
5 +-----+-----+-----+-----+
6 | 1 | 唐三藏 | 67 | 98 | 56 |
7 | 2 | 孙悟空 | 87 | 80 | 77 |
8 | 3 | 猪悟能 | 88 | 98 | 90 |
9 | 4 | 曹孟德 | 70 | 90 | 67 |
10 | 5 | 刘玄德 | 55 | 115 | 45 |
11 | 6 | 孙权   | 70 | 73 | 78 |
12 | 7 | 宋公明 | 75 | 95 | 30 |
13 | 8 | 张飞   | 27 | 0 | NULL |
14 +-----+-----+-----+-----+
15 8 rows in set (0.00 sec)
16
17 # 更新操作
18 mysql> update exam set chinese = chinese * 2;
19 Query OK, 8 rows affected (0.01 sec)
20 Rows matched: 8  Changed: 8  Warnings: 0
21
22 # 查看结果
23 mysql> select * from exam;
24 +-----+-----+-----+-----+
25 | id | name   | chinese | math | english |
26 +-----+-----+-----+-----+
27 | 1 | 唐三藏 | 134 | 98 | 56 |
28 | 2 | 孙悟空 | 174 | 80 | 77 |
29 | 3 | 猪悟能 | 176 | 98 | 90 |
30 | 4 | 曹孟德 | 140 | 90 | 67 |
31 | 5 | 刘玄德 | 110 | 115 | 45 |
32 | 6 | 孙权   | 140 | 73 | 78 |
33 | 7 | 宋公明 | 150 | 95 | 30 |
34 | 8 | 张飞   | 54 | 0 | NULL |
35 +-----+-----+-----+-----+
36 8 rows in set (0.00 sec)

```

5.3 Update 注意事项

- 以原值的基础上做变更时，不能使用math += 30这样的语法
- 不加where条件时，会导致全表数据被列新，谨慎操作

6. Delete 删除

6.1 语法

```
1 DELETE FROM tbl_name [WHERE where_condition] [ORDER BY ...] [LIMIT row_count]
```

6.2 示例

- 删除孙悟空同学的考试成绩

```
1 # 查看原始数据
2 mysql> select * from exam where name = '孙悟空';
3 +-----+-----+-----+-----+-----+
4 | id | name | chinese | math | english |
5 +-----+-----+-----+-----+-----+
6 | 2 | 孙悟空 | 174 | 80 | 77 |
7 +-----+-----+-----+-----+-----+
8 1 row in set (0.00 sec)
9
10 # 删除操作
11 mysql> delete from exam where name = '孙悟空';
12 Query OK, 1 row affected (0.01 sec)
13
14 # 查看结果
15 mysql> select * from exam where name = '孙悟空';
16 Empty set (0.00 sec)
```

- 删除整张表数据

```
1 # 准备测试表
2 mysql> CREATE TABLE t_delete (
3     id INT,
4     name VARCHAR(20)
5 );
6 Query OK, 0 rows affected (0.16 sec)
7
8 # 插入测试数据
9 mysql> INSERT INTO t_delete (id, name) VALUES (1, 'A'), (2, 'B'), (3, 'C');
10 Query OK, 3 rows affected (0.00 sec)
11 Records: 3 Duplicates: 0 Warnings: 0
12
13 # 查看测试表
14 mysql> select * from t_delete;
15 +-----+-----+
16 | id | name |
17 +-----+-----+
18 | 1 | A |
```

```

19 |      2 | B      |
20 |      3 | C      |
21 +-----+-----+
22 3 rows in set (0.00 sec)
23
24 # 删除整张表中的数据
25 mysql> delete from t_delete;
26 Query OK, 3 rows affected (0.00 sec)
27
28 # 查看结果
29 mysql> select * from t_delete;
30 Empty set (0.00 sec)

```

6.3 Delete注意事项

- 执行Delete时不加条件会删除整张表的数据，谨慎操作

7. 截断表

7.1 语法

```
1 TRUNCATE [TABLE] tbl_name
```

7.2 示例

```

1 # 准备测试表
2 mysql> CREATE TABLE t_truncate (
3     id INT PRIMARY KEY AUTO_INCREMENT,
4     name VARCHAR(20)
5 );
6 Query OK, 0 rows affected (0.16 sec)
7
8 # 插入测试数据
9 mysql> INSERT INTO t_truncate (name) VALUES ('A'), ('B'), ('C');
10 Query OK, 3 rows affected (1.05 sec)
11 Records: 3 Duplicates: 0 Warnings: 0
12
13 # 查看测试表
14 mysql> select * from t_truncate;
15 +-----+-----+
16 | id | name |

```

```

17 +-----+-----+
18 |  1  | A      |
19 |  2  | B      |
20 |  3  | C      |
21 +-----+-----+
22 3 rows in set (0.00 sec)
23
24 # 查看建表结构, AUTO_INCREMENT=4
25 mysql> show create table t_truncate;
26 ***** 1. row *****
27      Table: t_truncate
28 Create Table: CREATE TABLE `t_truncate` (
29   `id` int NOT NULL AUTO_INCREMENT,
30   `name` varchar(20) CHARACTER SET utf8mb4 DEFAULT NULL,
31   PRIMARY KEY (`id`)
32 ) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8mb4
   COLLATE=utf8mb4_0900_ai_ci |
33
34 1 row in set (0.00 sec)
35
36 # 截断表, 注意受影响的行数是0
37 mysql> truncate table t_truncate;
38 Query OK, 0 rows affected (0.01 sec)
39
40 # 查看表中的数据
41 mysql> select * from t_truncate;
42 Empty set (0.00 sec)
43
44 # 查看表结构, AUTO_INCREMENT已被重置为0
45 mysql> show create table t_truncate\G
46 ***** 1. row *****
47      Table: t_truncate
48 Create Table: CREATE TABLE `t_truncate` (
49   `id` int NOT NULL AUTO_INCREMENT,
50   `name` varchar(20) CHARACTER SET utf8mb4 DEFAULT NULL,
51   PRIMARY KEY (`id`)
52 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
53 1 row in set (0.00 sec)
54
55 # 继续写入数据
56 mysql> INSERT INTO t_truncate (name) VALUES ('D');
57 Query OK, 1 row affected (0.01 sec)
58
59 # 自增主键从1开始计数
60 mysql> select * from t_truncate;
61 +-----+-----+
62 | id | name |

```



```

63 +-----+-----+
64 | 1 | D |
65 +-----+-----+
66 1 row in set (0.00 sec)
67
68 # 再次查看表结构, AUTO_INCREMENT=2
69 mysql> show create table t_truncate\G
70 ***** 1. row *****
71      Table: t_truncate
72 Create Table: CREATE TABLE `t_truncate` (
73   `id` int NOT NULL AUTO_INCREMENT,
74   `name` varchar(20) CHARACTER SET gbk DEFAULT NULL,
75   PRIMARY KEY (`id`)
76 ) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4
77   COLLATE=utf8mb4_0900_ai_ci
78 1 row in set (0.00 sec)
79

```

7.3 Truncate注意事项

- 只能对整表操作, 不能像 DELETE 一样针对部分数据
- 不对数据操作所以比DELETE更快, TRUNCATE在删除数据的时候, 不经过真正的事物, 所以无法回滚
- 会重置 AUTO_INCREMENT 项

8. 插入查询结果

8.1 语法

```
1 INSERT INTO table_name [(column [, column ...])] SELECT ...
```

8.2 示例

- 删除表中的重复记录, 重复的数据只能有一份

```

1 # 创建测试表, 并构造数据
2 mysql> CREATE TABLE t_recored (id int, name varchar(20));
3
4 # 插入测试数据
5 INSERT INTO t_recored VALUES

```

```

6 (100, 'aaa'),
7 (100, 'aaa'),
8 (200, 'bbb'),
9 (200, 'bbb'),
10 (200, 'bbb'),
11 (300, 'ccc');
12
13 # 查看结果
14 mysql> select * from t_recored;
15 +-----+-----+
16 | id    | name |
17 +-----+-----+
18 | 100   | aaa  |
19 | 100   | aaa  |
20 | 200   | bbb  |
21 | 200   | bbb  |
22 | 200   | bbb  |
23 | 300   | ccc  |
24 +-----+-----+
25 6 rows in set (0.00 sec)

```

- **实现思路：**原始表中的数据一般不会主动删除，但是真正查询时不需要重复的数据，如果每次查询都使用DISTINCT进行去重操作，会严重效率。可以创建一张与 `t_recored` 表结构相同的表，把去重的记录写入到新表中，以后查询都从新表中查，这样真实的数据不丢失，同时又能保证查询效率

```

1 # 创建一张新表，表结构与t_recored相同
2 mysql> create table t_recored_new like t_recored;
3 Query OK, 0 rows affected (0.02 sec)
4
5 # 新表中没有记录
6 mysql> select * from t_recored_new;
7 Empty set (0.00 sec)
8
9 # 原表中的记录去重后写入到新表
10 mysql> insert into t_recored_new select distinct * from t_recored;
11 Query OK, 3 rows affected (0.00 sec)
12 Records: 3 Duplicates: 0 Warnings: 0
13
14 # 查询新表中的记录，实现去重
15 mysql> select * from t_recored_new;
16 +-----+-----+
17 | id    | name |
18 +-----+-----+
19 | 100   | aaa  |

```

```

20 | 200 | bbb |
21 | 300 | ccc |
22 +-----+-----+
23 3 rows in set (0.00 sec)
24
25 # 新表与原来重命名
26 mysql> rename table t_recored to t_recored_old, t_recored_new to t_recored;
27 Query OK, 0 rows affected (0.02 sec)
28
29 # 查询重命名后表中的记录，实现需求且原来中的记录不受影响
30 mysql> select * from t_recored;
31 +-----+-----+
32 | id   | name |
33 +-----+-----+
34 | 100  | aaa  |
35 | 200  | bbb  |
36 | 300  | ccc  |
37 +-----+-----+
38 3 rows in set (0.00 sec)
39
40 mysql> select * from t_recored_old;
41 +-----+-----+
42 | id   | name |
43 +-----+-----+
44 | 100  | aaa  |
45 | 100  | aaa  |
46 | 200  | bbb  |
47 | 200  | bbb  |
48 | 200  | bbb  |
49 | 300  | ccc  |
50 +-----+-----+
51 6 rows in set (0.00 sec)

```

9. 聚合函数

9.1 常用函数

函数	说明
COUNT([DISTINCT] expr)	返回查询到的数据的 数量
SUM([DISTINCT] expr)	返回查询到的数据的 总和，不是数字没有意义
AVG([DISTINCT] expr)	返回查询到的数据的 平均值，不是数字没有意义

MAX([DISTINCT] expr)	返回查询到的数据的 最大值，不是数字没有意义
MIN([DISTINCT] expr)	返回查询到的数据的 最小值，不是数字没有意义

9.2 示例

9.2.1 COUNT

- 统计exam表中有多少记录

```

1 # 使用 * 做统计
2 mysql> select count(*) from exam;
3 +-----+
4 | count(*) |
5 +-----+
6 |          7 |
7 +-----+
8 1 row in set (0.00 sec)
9
10 # 使用常量做统计
11 mysql> select count(1) from exam;
12 +-----+
13 | count(1) |
14 +-----+
15 |          7 |
16 +-----+
17 1 row in set (0.00 sec)

```

- 统计有多少学生参加数学考试

```

1 # # 使用指定列做统计
2 mysql> select count(math) from exam;
3 +-----+
4 | count(math) |
5 +-----+
6 |          7 |
7 +-----+
8 1 row in set (0.00 sec)

```

- 统计有多少学生参加英语考试

```

1 # 查看表中的记录

```

```

2 mysql> select * from exam;
3 +-----+-----+-----+-----+
4 | id | name   | chinese | math | english |
5 +-----+-----+-----+-----+
6 | 1 | 唐三藏 | 134    | 98   | 56   |
7 | 3 | 猪悟能 | 176    | 98   | 90   |
8 | 4 | 曹孟德 | 140    | 90   | 67   |
9 | 5 | 刘玄德 | 110    | 115  | 45   |
10 | 6 | 孙权   | 140    | 73   | 78   |
11 | 7 | 宋公明 | 150    | 95   | 30   |
12 | 8 | 张飞   | 54     | 0    | NULL | # 张飞没有参加英语考试
13 +-----+-----+-----+-----+
14 7 rows in set (0.00 sec)
15
16 # NULL 的数据不会计入结果
17 mysql> select count(english) from exam;
18 +-----+
19 | count(english) |
20 +-----+
21 | 6 |
22 +-----+
23 1 row in set (0.00 sec)

```

● 统计语文成绩小于50分的学生个数

```

1 # 加入where条件
2 mysql> select count(chinese) from exam where chinese < 10;
3 +-----+
4 | count(chinese) |
5 +-----+
6 | 0 |
7 +-----+
8 1 row in set (0.00 sec)

```

9.2.2 SUM

● 统计所有学生数学成绩总分

```

1 mysql> select sum(math) from exam;
2 +-----+
3 | sum(math) |
4 +-----+
5 | 569 |

```

```
6 +-----+
7 1 row in set (0.01 sec)
```

- 统计所有学生英语成绩总分

```
1 # 值为NULL的列不参与统计
2 mysql> select sum(english) from exam;
3 +-----+
4 | sum(english) |
5 +-----+
6 |          366 |
7 +-----+
8 1 row in set (0.00 sec)
```

- 不能统计非数值的列

```
1 mysql> select sum(name) from exam;
2 +-----+
3 | sum(name) |
4 +-----+
5 |          0 |
6 +-----+
7 1 row in set, 7 warnings (0.01 sec) # 警告信息, 可以使用show warnings查看
```

9.2.3 AVG

- 统计英语成绩的平均分

```
1 # NULL值不参与统计
2 mysql> select avg(english) from exam;
3 +-----+
4 | avg(english) |
5 +-----+
6 |          61 |
7 +-----+
8 1 row in set (0.00 sec)
```

- 统计平均总分

```

1 mysql> select avg(chinese + math + english) as 总分 from exam;
2 +-----+
3 | 总分 |
4 +-----+
5 | 297.5 |
6 +-----+
7 1 row in set (0.00 sec)

```

9.2.4 MAX

- 查询英语最高分

```

1 mysql> select max(english) from exam;
2 +-----+
3 | max(english) |
4 +-----+
5 |          90 |
6 +-----+
7 1 row in set (0.00 sec)

```

9.2.5 MIN

- 查询 > 70 分以上的数学最低分

```

1 mysql> select min(math) from exam where math > 70;
2 +-----+
3 | min(math) |
4 +-----+
5 |        73 |
6 +-----+
7 1 row in set (0.00 sec)

```

- 查询数据成绩的最高分与英语成绩的最低分

```

1 # 可以使用多个聚合函数
2 mysql> select max(math), min(english) from exam;
3 +-----+-----+
4 | max(math) | min(english) |
5 +-----+-----+
6 |        115 |          30 |
7 +-----+-----+
8 1 row in set (0.00 sec)

```

10. Group by 分组查询

GROUP BY 子句的作用是通过一定的规则将一个数据集划分成若干个小的分组，然后针对若干个分组进行数据处理，比如使用聚合函数对分组进行统计。

10.1 语法

```
1 SELECT {col_name | expr} ,... ,aggregate_function (aggregate_expr)
2     FROM table_references
3     GROUP BY {col_name | expr}, ...
4     [HAVING where_condition]
5
```

- **col_name | expr**: 要查询的列或表达式，可以有多个，必须在 **GROUP BY** 子句中作为分组的依据
- **aggregate_function**: 聚合函数，比如COUNT(), SUM(), AVG(), MAX(), MIN()
- **aggregate_expr**: 聚合函数传入的列或表达式，如果列或表达式**不在** **GROUP BY** 子句中，必须包含在聚合函数中

10.2 示例

- 准备测试表及数据职员表emp，列分别为：id(编号)，name(姓名)，role(角色)，salary(薪水)

```
1 drop table if exists emp;
2 create table emp (
3     id bigint primary key auto_increment,
4     name varchar(20) not null,
5     role varchar(20) not null,
6     salary decimal(10, 2) not null
7 );
8
9 insert into emp values (1, '马云', '老板', 1500000.00);
10 insert into emp values (2, '马化腾', '老板', 1800000.00);
11 insert into emp values (3, '鑫哥', '讲师', 10000.00);
12 insert into emp values (4, '博哥', '讲师', 12000.00);
13 insert into emp values (5, '平姐', '学管', 9000.00);
14 insert into emp values (6, '莹姐', '学管', 8000.00);
15 insert into emp values (7, '孙悟空', '游戏角色', 956.8);
16 insert into emp values (8, '猪悟能', '游戏角色', 700.5);
17 insert into emp values (9, '沙和尚', '游戏角色', 333.3);
```


18

```
19 select * from emp;
```

- 统计每个角色的人数

```
1 mysql> select role, count(*) from emp group by role;
2 +-----+-----+
3 | role      | count(*) |
4 +-----+-----+
5 | 老板      | 2        |
6 | 讲师      | 2        |
7 | 学管      | 2        |
8 | 游戏角色  | 3        |
9 +-----+-----+
10 4 rows in set (0.00 sec)
```

- 统计每个角色的平均工资，最高工资，最低工资

```
1 mysql> select role, ROUND(avg(salary),2) as 平均工资, ROUND(max(salary),2) as 最高工资, ROUND(min(salary),2) as 最低工资 from emp group by role;
2 +-----+-----+-----+-----+
3 | role      | 平均工资 | 最高工资 | 最低工资 |
4 +-----+-----+-----+-----+
5 | 老板      | 1650000.00 | 1800000.00 | 1500000.00 |
6 | 讲师      | 11000.00 | 12000.00 | 10000.00 |
7 | 学管      | 8500.00 | 9000.00 | 8000.00 |
8 | 游戏角色  | 663.53 | 956.80 | 333.30 |
9 +-----+-----+-----+-----+
10 4 rows in set (0.00 sec)
```

10.3 having子句

使用GROUP BY对结果进行分组处理之后，对分组的结果进行过滤时，不能使用WHERE子句，而要使用HAVING子句

- 显示平均工资低于1500的角色和它的平均工资

```
1 mysql> select role, avg(salary) from emp group by role having avg(salary) < 1500;
2 +-----+-----+
3 | role      | avg(salary) |
```

```

4 +-----+-----+
5 | 游戏角色 | 663.533333 |
6 +-----+-----+
7 1 row in set (0.00 sec)

```

10.4 Having 与Where 的区别

- Having 用于对分组结果的条件过滤
- Where 用于对表中真实数据的条件过滤

11. 内置函数

11.1 日期函数

函数	说明
<code>CURDATE()</code>	返回当前日期，同义词 <code>CURRENT_DATE</code> ， <code>CURRENT_DATE()</code>
<code>CURTIME()</code>	返回当前时间，同义词 <code>CURRENT_TIME</code> ， <code>CURRENT_TIME([fsp])</code>
<code>NOW()</code>	返回当前日期和时间，同义词 <code>CURRENT_TIMESTAMP</code> ， <code>CURRENT_TIMESTAMP</code>
<code>DATE(data)</code>	提取date或datetime表达式的日期部分
<code>ADDDATE(date,INTERVAL expr unit)</code>	向日期值添加时间值(间隔)，同义词 <code>DATE_ADD()</code>
<code>SUBDATE(date,INTERVAL expr unit)</code>	向日期值减去时间值(间隔)，同义词 <code>DATE_SUB()</code>
<code>DATEDIFF(expr1,expr2)</code>	两个日期的差，以天为单位， <code>expr1 - expr2</code>

11.1.1 示例

- 获取当前日期

```

1 mysql> select CURDATE();
2 +-----+
3 | CURDATE() |
4 +-----+
5 | 2024-08-27 |
6 +-----+

```

```
7 1 row in set (0.00 sec)
```

- 获取当前时间

```
1 mysql> select CURTIME();
2 +-----+
3 | CURTIME() |
4 +-----+
5 | 10:26:33 |
6 +-----+
7 1 row in set (0.00 sec)
```

- 获取当前日期和时间

```
1 mysql> select NOW();
2 +-----+
3 | NOW() |
4 +-----+
5 | 2024-08-27 10:26:47 |
6 +-----+
7 1 row in set (0.00 sec)
```

- 提取指定datetime的日期部分

```
1 mysql> select date('2024-08-27 10:26:47');
2 +-----+
3 | date('2024-08-27 10:26:47') |
4 +-----+
5 | 2024-08-27 |
6 +-----+
7 1 row in set (0.00 sec)
```

- 在给定日期的基础上加31天

```
1 mysql> select ADDDATE('2024-01-02', INTERVAL 31 DAY);
2 +-----+
3 | ADDDATE('2024-01-02', INTERVAL 31 DAY) |
4 +-----+
5 | 2024-02-02 |
```

```
6 +-----+
7 1 row in set (0.00 sec)
```

- 在给定日期的基础上减去1月

```
1 mysql> select SUBDATE('2024-08-02', INTERVAL 1 MONTH);
2 +-----+
3 | SUBDATE('2024-08-02', INTERVAL 1 MONTH) |
4 +-----+
5 | 2024-07-02 |
6 +-----+
7 1 row in set (0.00 sec)
```

- 计算两个日期之间相差多少天

```
1 # 在计算时只使用日期部分
2 mysql> SELECT DATEDIFF('2024-12-31 23:59:59', '2024-12-30');
3 +-----+
4 | DATEDIFF('2024-12-31 23:59:59', '2024-12-30') |
5 +-----+
6 | 1 |
7 +-----+
8 1 row in set (0.00 sec)
9
10 # 表达式1表示的日期早于表达式2表示的日期时返回负数
11 mysql> SELECT DATEDIFF('2024-11-30 23:59:59', '2024-12-31');
12 +-----+
13 | DATEDIFF('2024-11-30 23:59:59', '2024-12-31') |
14 +-----+
15 | -31 |
16 +-----+
17 1 row in set (0.00 sec)
```

11.1.2 参考链接

<https://dev.mysql.com/doc/refman/8.0/en/date-and-time-functions.html>

14.7 Date and Time Functions

Here is an example that uses date functions. The following query selects all rows with a date_col value from within the last 30 days: The query also selects rows with dates that lie in the future. Fun

11.5 Expressions

The following grammar rules define expression syntax in MySQL. The grammar shown here is based on that given in the sql/sql_yacc.yy file of MySQL source distributions. For additional information about

11.2 字符串处理函数

函数	说明
<code>CHAR_LENGTH(str)</code>	返回给定字符串的长度，同义词 <code>CHARACTER_LENGTH()</code>
<code>LENGTH(str)</code>	返回给定字符串的字节数，与当前使用的字符编码集有关
<code>CONCAT(str1,str2,...)</code>	返回拼接后的字符串
<code>CONCAT_WS(separator,str1,str2,...)</code>	返回拼接后带分隔符的字符串
<code>LCASE(str)</code>	将给定字符串转换成小写，同义词 <code>LOWER()</code>
<code>UCASE(str)</code>	将给定字符串转换成大写，同义词 <code>UPPER()</code>
<code>HEX(str)</code> , <code>HEX(N)</code>	对于字符串参数str, <code>HEX()</code> 返回str的十六进制字符串表示形式，对于数字参数N, <code>HEX()</code> 返回一个十六进制字符串表示形式
<code>INSTR(str,substr)</code>	返回substring第一次出现的索引
<code>INSERT(str,pos,len,newstr)</code>	在指定位置插入子字符串，最多不超过指定的字符数
<code>SUBSTR(str,pos)</code> <code>SUBSTR(str FROM pos FOR len)</code>	返回指定的子字符串，同义词 <code>SUBSTRING(str,pos)</code> , <code>SUBSTRING(str FROM pos FOR len)</code>
<code>REPLACE(str,from_str,to_str)</code>	把字符串str中所有的from_str替换为to_str，区分大小写
<code>STRCMP(expr1,expr2)</code>	逐个字符比较两个字符串，返回 -1, 0, 1
<code>LEFT(str,len)</code> , <code>RIGHT(str,len)</code>	返回字符串str中最左/最右边的len个字符
<code>LTRIM(str)</code> , <code>RTRIM(str)</code> , <code>TRIM(str)</code>	删除给定字符串的前导、末尾、前导和末尾的空格
<code>TRIM([{LEADING TRAILING BOTH } [remstr] FROM</code>	删除给定字符串的前导、末尾或前导和末尾的指定字符串

11.2.1 示例

- 显示所有参加考试的学生姓名、姓名字符数和字节长度

```
1 mysql> select name, char_length(name), length(name) from exam;
2 +-----+-----+-----+
3 | name      | char_length(name) | length(name) |
4 +-----+-----+-----+
5 | 唐三藏    | 3                 | 6            |
6 | 猪悟能    | 3                 | 6            |
7 | 曹孟德    | 3                 | 6            |
8 | 刘玄德    | 3                 | 6            |
9 | 孙权      | 2                 | 4            |
10 | 宋公明    | 3                 | 6            |
11 | 张飞      | 2                 | 4            |
12 +-----+-----+-----+
13 7 rows in set (0.00 sec)
```

- 显示学生的考试成绩，格式为 "XXX的语文成绩：XXX分，数学成绩：XXX分，英语成绩：XXX分"

```
1 mysql> select concat(name, '的语文成绩：', chinese, '分，数学成绩：', math, '分，英
   语成绩：', english, '分') as 分数 from exam;
2 +-----+
3 | 分数
4 +-----+
5 | 唐三藏的语文成绩：134分，数学成绩：98分，英语成绩：56分
6 | 猪悟能的语文成绩：176分，数学成绩：98分，英语成绩：90分
7 | 曹孟德的语文成绩：140分，数学成绩：90分，英语成绩：67分
8 | 刘玄德的语文成绩：110分，数学成绩：115分，英语成绩：45分
9 | 孙权的语文成绩：140分，数学成绩：73分，英语成绩：78分
10 | 宋公明的语文成绩：150分，数学成绩：95分，英语成绩：30分
11 | NULL
12 +-----+
13 7 rows in set (0.00 sec)
```

- 拼接后的字符串用逗号隔开

```

1 mysql> select CONCAT_WS(',',name,chinese,math,english) 分数 from exam;
2 +-----+
3 | 分数          |
4 +-----+
5 | 唐三藏,134,98,56 |
6 | 猪悟能,176,98,90 |
7 | 曹孟德,140,90,67 |
8 | 刘玄德,110,115,45 |
9 | 孙权,140,73,78   |
10 | 宋公明,150,95,30 |
11 | 张飞,54,0        |
12 +-----+
13 7 rows in set (0.00 sec)

```

- 将给定字符串转换成小写

```

1 mysql> select lcase('ABC');
2 +-----+
3 | lcase('ABC') |
4 +-----+
5 | abc          |
6 +-----+
7 1 row in set (0.00 sec)

```

- 将给定字符串转换成小写

```

1 mysql> select ucase('abc');
2 +-----+
3 | ucase('abc') |
4 +-----+
5 | ABC          |
6 +-----+
7 1 row in set (0.00 sec)

```

- 转换为十六进制

```

1 # 字符串
2 mysql> select HEX('Hello MySQL');
3 +-----+
4 | HEX('Hello MySQL') |

```

```

5 +-----+
6 | 48656C6C6F204D7953514C |
7 +-----+
8 1 row in set (0.00 sec)
9
10 # 数字
11 mysql> select HEX(15);
12 +-----+
13 | HEX(15) |
14 +-----+
15 | F       |
16 +-----+
17 1 row in set (0.00 sec)

```

• 子字符串第一次出现的索引

```

1 mysql> select instr('Hello MySQL', 'sql');
2 +-----+
3 | instr('Hello MySQL', 'sql') |
4 +-----+
5 |                               9 |
6 +-----+
7 1 row in set (0.00 sec)

```

• 指定位置插入子字符串

```

1 # 在指定位置插入
2 mysql> select insert('Hello, Database', 8, 0, 'MySQL ');
3 +-----+
4 | insert('Hello, Database', 8, 0, 'MySQL ') |
5 +-----+
6 | Hello, MySQL Database                      |
7 +-----+
8 1 row in set (0.00 sec)
9
10 # 覆盖20位，如果从写入点往后不足20位相当于删除后面所有字符
11 mysql> select insert('Hello, Database', 8, 20, 'MySQL');
12 +-----+
13 | insert('Hello, Database', 8, 20, 'MySQL') |
14 +-----+
15 | Hello, MySQL                               |
16 +-----+
17 1 row in set (0.00 sec)

```


- 返回指定的子字符串

```
1 # 从'Hello, MySQL'的第八个字符开始截取
2 mysql> select SUBSTR('Hello, MySQL', 8);
3 +-----+
4 | SUBSTR('Hello, MySQL', 8) |
5 +-----+
6 | MySQL                      |
7 +-----+
8 1 row in set (0.00 sec)
9
10 # 从'Hello, MySQL'的第八个字符开始截取1个字符
11 mysql> select SUBSTR('Hello, MySQL' FROM 8 FOR 1);
12 +-----+
13 | SUBSTR('Hello, MySQL' FROM 8 FOR 1) |
14 +-----+
15 | M                                   |
16 +-----+
17 1 row in set (0.00 sec)
18
19 # 从'Hello, MySQL'的第八个字符开始截取10个字符，不足10个读到整个字符串结尾
20 mysql> select SUBSTR('Hello, MySQL' FROM 8 FOR 10);
21 +-----+
22 | SUBSTR('Hello, MySQL' FROM 8 FOR 10) |
23 +-----+
24 | MySQL                               |
25 +-----+
26 1 row in set (0.00 sec)
```

- 替换字符串

```
1 # 把Database替换成MySQL，区分大小写
2 mysql> select REPLACE('Hello Database', 'Database', 'MySQL');
3 +-----+
4 | REPLACE('Hello Database', 'Database', 'MySQL') |
5 +-----+
6 | Hello MySQL                                   |
7 +-----+
8 1 row in set (0.00 sec)
```

- 比较两个字符串

```

1 # 观察返回的结果
2 mysql> select STRCMP('text', 'text1');
3 +-----+
4 | STRCMP('text', 'text1') |
5 +-----+
6 |                        -1 |
7 +-----+
8 1 row in set (0.00 sec)
9
10 mysql> select STRCMP('text', 'text');
11 +-----+
12 | STRCMP('text', 'text') |
13 +-----+
14 |                        0 |
15 +-----+
16 1 row in set (0.00 sec)
17
18 mysql> select STRCMP('text1', 'text');
19 +-----+
20 | STRCMP('text1', 'text') |
21 +-----+
22 |                        1 |
23 +-----+
24 1 row in set (0.00 sec)

```

- 返回字符串str中最左/最右边的len个字符

```

1 # 最左边的5个字符
2 mysql> select LEFT('Hello MySQL', 5);
3 +-----+
4 | LEFT('Hello MySQL', 5) |
5 +-----+
6 | Hello                   |
7 +-----+
8 1 row in set (0.00 sec)
9
10 # 最右边的5个字符
11 mysql> select RIGHT('Hello MySQL', 5);
12 +-----+
13 | RIGHT('Hello MySQL', 5) |
14 +-----+
15 | MySQL                   |
16 +-----+
17 1 row in set (0.00 sec)

```

- 删除给定字符串的前导、末尾、前导和末尾的空格

```

1 mysql> select '   ABC', LTRIM('   ABC'), 'ABC ', RTRIM('ABC '), '   ABC
2 +-----+-----+-----+-----+-----+-----+
3 | ABC | LTRIM('   ABC') | ABC | RTRIM('ABC ') | ABC | TRIM('
4 +-----+-----+-----+-----+-----+-----+
5 |   ABC | ABC | ABC | ABC | ABC | ABC
6 +-----+-----+-----+-----+-----+-----+
7 1 row in set (0.00 sec)

```

- 删除给定字符串的前导、末尾或前导和末尾的指定字符串

```

1 # 删除前后指定的字符串, BOTH可以省略
2 mysql> select TRIM(BOTH 'xxx' FROM 'xxxABCxxx');
3 +-----+
4 | TRIM('xxx' FROM 'xxxABCxxx') |
5 +-----+
6 | ABC |
7 +-----+
8 1 row in set (0.00 sec)
9
10 # 删除前导的指定字符串
11 mysql> select TRIM(LEADING 'xxx' FROM 'xxxABCxxx');
12 +-----+
13 | TRIM(LEADING 'xxx' FROM 'xxxABCxxx') |
14 +-----+
15 | ABCxxx |
16 +-----+
17 1 row in set (0.00 sec)
18
19 # 删除末尾的指定字符串
20 mysql> select TRIM(TRAILING 'xxx' FROM 'xxxABCxxx');
21 +-----+
22 | TRIM(TRAILING 'xxx' FROM 'xxxABCxxx') |
23 +-----+
24 | xxxABC |
25 +-----+
26 1 row in set (0.00 sec)

```

11.2.2 参考链接

<https://dev.mysql.com/doc/refman/8.0/en/string-functions.html>

14.8 String Functions and Operators

interprets each argument as an integer and returns a string consisting of the characters given by the code values of those integers. returns a binary string. To produce a string in a given character s

11.3 数学函数

函数	说明
<code>ABS(X)</code>	返回X的绝对值
<code>CEIL(X)</code>	返回不小于X的最小整数值，同义词是 <code>CEILING(X)</code>
<code>FLOOR(X)</code>	返回不大于X的最大整数值
<code>CONV(N,from_base,to_base)</code>	不同进制之间的转换
<code>FORMAT(X,D)</code>	将数字X格式化为“#，###，###”的格式。##'，四舍五入到小数点后D位，并以字符串形式返回
<code>RAND([N])</code>	返回一个随机浮点值，取值范围 [0.0, 1.0)
<code>ROUND(X)</code> , <code>ROUND(X,D)</code>	将参数X舍入到小数点后D位
<code>CRC32(expr)</code>	计算指定字符串的循环冗余校验值并返回一个32位无符号整数

11.3.1 示例

- 返回-3.14的绝对值

```
1 mysql> select ABS(-3.14);
2 +-----+
3 | ABS(-3.14) |
4 +-----+
5 |      3.14 |
6 +-----+
7 1 row in set (0.00 sec)
```

- 返回不小于20.36的最小整数值

```

1 mysql> select CEIL(20.36);
2 +-----+
3 | CEIL(20.36) |
4 +-----+
5 |          21 |
6 +-----+
7 1 row in set (0.00 sec)

```

- 返回不大于11.32的最小整数值

```

1 mysql> select FLOOR(11.32);
2 +-----+
3 | FLOOR(11.32) |
4 +-----+
5 |          11 |
6 +-----+
7 1 row in set (0.00 sec)

```

- 10进制转为16进制

```

1 mysql> select CONV(15, 10, 16);
2 +-----+
3 | CONV(15, 10, 16) |
4 +-----+
5 | F                |
6 +-----+
7 1 row in set (0.00 sec)

```

- 格式化1234567.654321

```

1 mysql> select FORMAT(1234567.654321, 5);
2 +-----+
3 | FORMAT(1234567.654321, 5) |
4 +-----+
5 | 1,234,567.65432          |
6 +-----+
7 1 row in set (0.00 sec)

```

- 返回一个随机浮点值

```

1 mysql> select RAND();
2 +-----+
3 | RAND() |
4 +-----+
5 | 0.9555353624332095 |
6 +-----+
7 1 row in set (0.00 sec)

```

- 舍弃到小数点后6位

```

1 mysql> select ROUND(RAND(), 6);
2 +-----+
3 | ROUND(RAND(), 6) |
4 +-----+
5 | 0.612703 |
6 +-----+
7 1 row in set (0.00 sec)
8
9 # 生成一个6位数的随机数
10 mysql> select ROUND(RAND(), 6) * 1000000;
11 +-----+
12 | ROUND(RAND(), 6) * 1000000 |
13 +-----+
14 | 982956 |
15 +-----+
16 1 row in set (0.00 sec)

```

- 字符串的循环冗余校验

```

1 mysql> select CRC32('Hello MySQL');
2 +-----+
3 | CRC32('Hello MySQL') |
4 +-----+
5 | 1944582821 |
6 +-----+
7 1 row in set (0.00 sec)

```

11.3.2 参考链接

<https://dev.mysql.com/doc/refman/8.0/en/numeric-functions.html>

MySQL :: MySQL 8.0 Reference Manual :: 14.6 Numeric Functions and Operators

Return the largest integer value not greater than the argument

11.4 其他常用函数

函数	说明
<code>version()</code>	显示当前数据库版本
<code>database()</code>	显示当前正在使用的数据库
<code>user()</code>	显示当前用户
<code>md5(str)</code>	对一个字符串进行md5摘要，摘要后得到一个32位字符串
<code>ifnull(val1, val2)</code>	如果val1为NULL，返回val2，否则返回 val1

11.4.1 示例

- 显示当前数据库版本

```
1 mysql> select version();
2 +-----+
3 | version() |
4 +-----+
5 | 8.0.39    |
6 +-----+
7 1 row in set (0.00 sec)
```

- 显示当前正在使用的数据库

```
1 # 没有选择数据库时
2 mysql> select database();
3 +-----+
4 | database() |
5 +-----+
6 | NULL      |
7 +-----+
8 1 row in set (0.00 sec)
9
10 # 选择数据库
```

```

11 mysql> use java01
12 Database changed
13
14 # 选择数据库后
15 mysql> select database();
16 +-----+
17 | database() |
18 +-----+
19 | java01      |
20 +-----+
21 1 row in set (0.00 sec)

```

• 显示当前用户

```

1 mysql> select user();
2 +-----+
3 | user()      |
4 +-----+
5 | root@localhost |
6 +-----+
7 1 row in set (0.00 sec)

```

• 对一个字符串进行md5加密

```

1 # 对hello world进行md5加密
2 mysql> select md5('hello world');
3 +-----+
4 | md5('hello world') |
5 +-----+
6 | 5eb63bbbe01eeed093cb22bb8f5acdc3 |
7 +-----+
8 1 row in set (0.01 sec)

```

• ifnull函数

```

1 # 第一个参数不为NULL， 返回第一个参数的值
2 mysql> select ifnull('database', 'MySQL');
3 +-----+
4 | ifnull('database', 'MySQL') |
5 +-----+
6 | database                     |

```



```
7 +-----+
8 1 row in set (0.00 sec)
9
10 # 第一个参数为NULL， 返回第二个参数的值
11 mysql> select ifnull(NULL, 'MySQL');
12 +-----+
13 | ifnull(NULL, 'MySQL') |
14 +-----+
15 | MySQL                  |
16 +-----+
17 1 row in set (0.00 sec)
```